Functional Programming / Lean

Type Classes — Overloading in Lean

Prof. Dr. Peter Thiemann

Albert-Ludwigs-Universität Freiburg, Germany

WS 2026/27

Overloading

Remember previous classes?

We were able to use

- equality == and ordering < with many different types
- arithmetic operations with many different types

Overloading

Remember previous classes?

We were able to use

- equality == and ordering < with many different types
- arithmetic operations with many different types

Overloading

The **same operator** can be used to execute **different code** at **many different types**.

Overloading

Remember previous classes?

We were able to use

- equality == and ordering < with many different types
- arithmetic operations with many different types

Overloading

The **same operator** can be used to execute **different code** at **many different types**.

Contrast with

Overloading

Remember previous classes?

We were able to use

- equality == and ordering < with many different types
- arithmetic operations with many different types

Overloading

The **same operator** can be used to execute **different code** at **many different types**.

Contrast with

Parametric polymorphism

The **same code** can execute at **many different types**.

Lean integrates overloading with polymorphism

Constrained polymorphism

- Some functions work on parametric types, but are constrained to specific instances
- Types contain type variables and **constraints**

Lean integrates overloading with polymorphism

Constrained polymorphism

- Some functions work on parametric types, but are constrained to specific instances
- Types contain type variables and **constraints**

Examples

```
-- elem x xs : is x an element of list xs?
-- type a must have equality
List.elem {α : Type} [BEq α] (a : α) (l : List α) : Bool
-- insertSorted x xs : insert x into sorted list xs
-- type a must have comparison
insertSorted : [LE α] → [DecidableLE α] → α → List α → List α
-- square x : compute the square of x
-- type α has multiplication
square : [Mul α] → α → α
```

Type classes

- Each constraint mentions a **type class**
  like `BEq`, `LE`, `Mul`, ...
- A type class specifies a set of operations for a type
  e.g. `BEq` requires an operator beq
- Type classes form a hierarchy
  e.g. `LawfulBEq` (α : `Type`) [`BEq` α] : `Prop` **extends** `ReflBEq` α
- Many classes are predefined, but you can roll your own

Classes and Instances

- A *class declaration* specifies a signature (i.e., the class members and their types)

```
class Monoid (α : Type) where
  op : α → α → α
  id : α
```

- An *instance declaration* specifies that a type belongs to a class by giving definitions for all class members

```
instance MonoidInt : Monoid Int where
  op := Int.add
  id := 0
```

- This info can be obtained from Lean by

```
#print Monoid
```

Defining `Monoid` for pairs

```
instance MonoidPair [Monoid α][Monoid β] : Monoid (α × β) where
  op := fun (x1, y1) (x2, y2) => (Monoid.op x1 x2, Monoid.op y1 y2)
  id := (Monoid.id, Monoid.id)
```

- Instance can have any number of assumptions / constraints on the type variables.
- Definition is **not** recursive!

Groups extend monoids

```
1 class Group (α : Type) extends Monoid α where
2   inv : α → α
3
4 instance GroupInt : Group Int where
5   inv := Int.neg
6
7 instance GroupPair [Group α][Group β] : Group (α × β) where
8   inv := fun (x, y) => (Group.inv x, Group.inv y)
```

- requires the `MonoidInt` instance
- requires the `MonoidPair` instance
- extension means:
  can use instances of `Group` where instances of `Monoid` is required

Further useful classes

ToString and Repr

```
class ToString (α : Type) where
  toString : α → String

class Repr (α : Type) where
  reprPrec : α → Nat → Std.Format
```

- `Repr` is predefined for most built-in types
- Derivable for most datatype definitions

Handwriting vs deriving an instance

Remember the Hearts game

```
1  inductive Color where
2    | red
3    | black
4
5  instance : ToString Color where
6    toString
7      | .red => "red"
8      | .black => "black"
9
10 instance : BEq Color where
11   beq
12     | .red, .red => true
13     | .black, .black => true
14     | _, _ => false
```

The `Ord` class (derivable)

```
1 inductive Ordering where
2   | lt | eq | gt
3
4 class Ord (α : Type) where
5   compare : α → α → Ordering
```

Different from `LE` and `LT`!

Equality

The type class BEq

```
class BEq (α : Type) where
  beq : α → α -> Bool
infix:50 " == " => BEq.beq

def bne [BEq α] (x y : α) : Bool :=
  !(x == y)
infix:50 " != " => bne
```

- Equality is special because Lean also supports proof
- Distinguish
  - *boolean equality* `beq` for programming
  - *propositional equality* $=$ for proving
- They are connected

Logical backing for equality

```
class ReflBEq (α) [BEq α] : Prop where
  protected rfl {a : α} : a == a
```

- Constrained to members of `BEq`
- Contains the logical proposition $\forall a.a == a$ (reflexivity)
- Instantiating `ReflBEq` requires *proof* of this proposition.

```
class LawfulBEq (α : Type u) [BEq α] : Prop extends ReflBEq α where
  eq_of_beq : {a b : α} → a == b → a = b
```

- extends: Can use `LawfulBEq` instances where `ReflBEq` is required
- `eq_of_beq` says that boolean equality implies propositional equality.
- In this context `a == b` stands for `(a == b) = true`!
- Instantiating also requires proof.

Equality

Flavors of equality ...

What does $a = b$ mean?

Equality

Flavors of equality ...

What does $a = b$ mean?

Propositional equality

There is a proof that a and b are equal.

Equality

Flavors of equality ...

What does $a = b$ mean?

Propositional equality

There is a proof that a and b are equal.

Definitional equality

a and b evaluate to the same result.
(by applying definitions)
Here the proof for $a = b$ is by *reflexivity*.

Equality

Flavors of equality ...

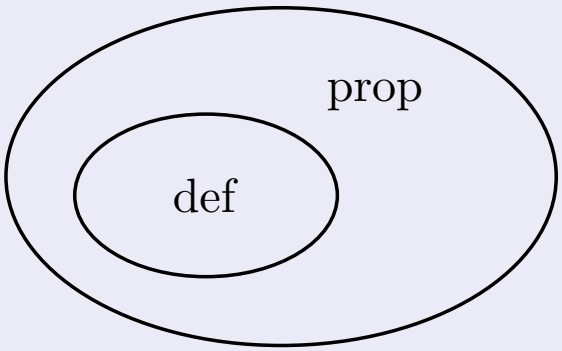
What does $a = b$ mean?

Propositional equality

There is a proof that a and b are equal.

Definitional equality

a and b evaluate to the same result.
(by applying definitions)
Here the proof for $a = b$ is by *reflexivity*.



Defining propositional equality

Another inductive datatype

```
inductive Eq (x : α) : α → Prop where
  | refl : Eq x x
```

Propositional equality is an equivalence relation

```
theorem symEq : ∀ {x y : α}, Eq x y → Eq y x :=
  fun h =>
  match h with
  | Eq.refl => Eq.refl

theorem transEq : ∀ {x y z : α}, Eq x y → Eq y z → Eq x z :=
  fun h1 h2 =>
  match h1, h2 with
  | Eq.refl, Eq.refl => Eq.refl
```

Lean wraps the `Eq.refl` constructor in `rfl`.

Excursion on proofs

Propositions and proofs

Examples

```
2 * 3 = 6
Sorted [1,2,3]
insertSorted 2 [1, 3] = [1, 2, 3]
```

Excursion on proofs

Propositions and proofs

Examples

```
2 * 3 = 6
Sorted [1,2,3]
insertSorted 2 [1, 3] = [1, 2, 3]
```

Proof by tactic

```
theorem easyDef1 : 4 + 5 = 9 := by rfl
```

`4 + 5` and `9` are *definitionally equal*, so the proposition is true by reflexivity of equality. Using tactic `rfl` preceded by keyword **by**.

Excursion on proofs

Propositions and proofs

Examples

```
2 * 3 = 6
Sorted [1,2,3]
insertSorted 2 [1, 3] = [1, 2, 3]
```

Proof by tactic

```
theorem easyDef1 : 4 + 5 = 9 := by rfl
```

`4 + 5` and `9` are *definitionally equal*, so the proposition is true by reflexivity of equality. Using tactic `rfl` preceded by keyword **by**.

Proof by proof term

```
theorem easyDef0 : 2 * 3 = 6 := eq_of_beq (BEq.refl 6)
```

- `BEq.refl 6` gives `(6 == 6) = true`
- `eq_of_beq` yields the required `6 = 6`

Defining propositions

Propositions behave like values

A provable proposition

```
1 def prop1 : Prop := 3 + 4 = 7
2 theorem prop1_is_true : prop1 := rfl
```

Defining propositions

Propositions behave like values

A provable proposition

```
1 def prop1 : Prop := 3 + 4 = 7
2 theorem prop1_is_true : prop1 := rfl
```

A false proposition

```
1 def prop2 : Prop := 3 + 4 = 8
```

Defining propositions

Propositions behave like values

A provable proposition

```
1 def prop1 : Prop := 3 + 4 = 7
2 theorem prop1_is_true : prop1 := rfl
```

A false proposition

```
1 def prop2 : Prop := 3 + 4 = 8
```

Another provable proposition

```
1 theorem prop3_thm : 3 ≤ 4 := by
2   decide
```

Tactic `decide` invokes a *decision procedure* that (in this case) constructs a proof for $3 \leq 4$. It only works with concrete arguments.

Building blocks of propositions

The logical connectives

"And" and "Or" definable using inductives

```
inductive And (p q : Prop) : Prop where
  | intro : p → q → And p q

inductive Or (p q : Prop) : Prop where
  | inl : p → Or p q
  | inr : q → Or p q
```

There are syntax definitions for $p \wedge q$ and $p \vee q$.

```
theorem and1 : 1 + 1 = 2 ∧ 2 + 2 = 4 := by decide  -- rfl does not work
theorem or1 : 1 + 1 = 2 ∨ 2 + 2 = 5 := by decide  -- rfl does not work
theorem and2 : 1 + 1 = 2 ∧ 2 + 2 ≤ 5 := by decide
```

Combinations of decidable propositions are also decidable.

Some simple theorems

Proof by giving a proof term of suitable type

```
1 theorem and_comm {p q : Prop} : p ∧ q → q ∧ p :=
2   fun pq =>
3   match pq with
4   | .intro left right => .intro right left
```

Some simple theorems

Proof by giving a proof term of suitable type

```
1 theorem and_comm {p q : Prop} : p ∧ q → q ∧ p :=
2   fun pq =>
3   match pq with
4   | .intro left right => .intro right left
```

Proof by tactics

```
1 theorem and_comm' {p q : Prop} : p ∧ q → q ∧ p := by
2   intro pq
3   exact .intro pq.right pq.left
```

- Tactic `intro pq` makes `p ∧ q` an assumption and `q ∧ p` the goal
- Tactic `exact` switches to proof term mode

Application: Indexing

Example

```
def bassists : List String := ["Geddy Lee", "John Myung", "Chris Squire"]

def john_myung : String := bassists[1]
#eval john_myung
def chris_squire : String := bassists[2]
#eval chris_squire
def morty : String := bassists[3] -- Error because index is out of bounds
#eval! morty
```

Application: Indexing

Example

```
1 def bassists : List String := ["Geddy Lee", "John Myung", "Chris Squire"]
2
3 def john_myung : String := bassists[1]
4 #eval john_myung
5 def chris_squire : String := bassists[2]
6 #eval chris_squire
7 def morty : String := bassists[3] -- Error because index is out of bounds
8 #eval! morty
```

Error

```
failed to prove index is valid, possible solutions:
  - Use ‘have‘-expressions to prove the index is valid
  - Use ‘a[i]!‘ notation instead, runtime check is performed, and ’
  - Use ‘a[i]?‘ notation instead, result is an ‘Option‘ type
  - Use ‘a[i]’h‘ notation instead, where ‘h‘ is a proof that index
|- 3 < bassists.lengthLean 4
```

Safe Indexing

Not accepted

```
1 def third {α } (xs : List α) : α := xs[2]
```

Safe Indexing

Not accepted

```
1 def third {α } (xs : List α) : α := xs[2]
```

Ok

```
1 def third {α} (xs : List α) (h : xs.length > 2) : α := xs[2]
2
3 #eval third bassists (by decide)
```

Inductive Propositions

An inductive proposition

```
inductive Sorted [LE α] : List α → Prop where
  | nil : Sorted []
  | cons : ∀ {x xs}, Sorted xs → (∀ y ∈ xs, x ≤ y) → Sorted (x :: xs)

def Sorted123 : Prop := Sorted [1, 2, 3]
```

Wrapup

Type classes

- provide a signature for an abstract data type
- instances provide implementations at unrelated types
- many classes are predefined and derivable
- pervasively used in Lean